Demo Guide — RHOAI 3.5 Models-as-a-Service (~15 minutes)

Run `make preflight` before presenting. Run `make status` for live readiness.

Console: `https://console-openshift-console.apps.<cluster-domain>`
MaaS URL: `https://maas.apps.<cluster-domain>` (printed by `make status`)

A. Preflight — cluster readiness (2 min)

WHERE I CLICK	Terminal → `make preflight`
WHAT I SHOW	PASS lines for OpenShift, RHOAI, cluster health; MISSING lines for MaaS prerequisites on a fresh sandbox
WHAT I SAY	"Before we install anything, we verify the cluster is healthy and list what MaaS still needs. On a new sandbox, Gateway, Kuadrant, and PostgreSQL are expected to be missing."
EXPECTED RESULT	`SAFE TO CONTINUE: YES`

B. Bootstrap — platform + MaaS (3 min)

WHERE I CLICK	Terminal → `make bootstrap`
WHAT I SHOW	Phases: GatewayClass, Kuadrant, Authorino TLS, PostgreSQL, `maas-db-config`, MaaS API/controller, DSC `modelsAsService=Managed`, demo namespace + simulator
WHAT I SAY	"One idempotent script converges the full MaaS stack: gateway in openshift-ingress, Kuadrant for policy, Authorino for auth, PostgreSQL for MaaS metadata, and our CPU demo model registered with the tenant."
EXPECTED RESULT	Script exits 0; `make status` shows Gateway Programmed, Kuadrant Ready, maas-api ready

---

C. Dashboard — MaaS in OpenShift AI (2 min)

WHERE I CLICK	OpenShift AI → **Governance** (or **MaaS**) → **Subscriptions** / **Auth policies**
WHAT I SHOW	`maas-demo-subscription`, `maas-demo-access` auth policy
WHAT I SAY	"MaaS separates who can call which model: subscriptions grant access, auth policies enforce it at the gateway via Authorino."
EXPECTED RESULT	Subscription and policy objects present in `models-as-a-service`

D. Demo model (1 min)

WHERE I CLICK	OpenShift AI → **Projects** → `rhoai-maas-demo` → **Models**
WHAT I SHOW	`demo-simulator` — CPU-backed `LLMInferenceService`, Ready
WHAT I SAY	"We use a lightweight CPU simulator so this demo does not compete for GPU with production models like Llama."
EXPECTED RESULT	Pod Running, `MaaSModelRef` phase Available

E. Inference via API (3 min)

WHERE I CLICK	Terminal → `make infer`
WHAT I SHOW	OpenAI-compatible request to `https://maas.apps.<domain>/v1/chat/completions` with Bearer API key
WHAT I SAY	"Clients use the same OpenAI SDK they already know. The gateway terminates TLS, Authorino validates the key, Kuadrant applies rate limits, and MaaS routes to the registered model."
EXPECTED RESULT	HTTP 200, JSON chat completion response

F. Access control demo (2 min)

WHERE I CLICK	Terminal → `make demo-auth`
WHAT I SHOW	Step 1: HTTP 200 with policy → Step 2: delete `MaaSAuthPolicy` → HTTP **403** → Step 3: restore policy → HTTP **200**
WHAT I SAY	"A valid subscription alone is not enough. Without the auth policy, Authorino denies the call — this is governed access, not an open proxy."
EXPECTED RESULT	`403-on-remove = true`, `200-on-restore = true`

G. Rate limiting demo (2 min)

WHERE I CLICK	Terminal → `make demo-rate-limit`
WHAT I SHOW	Burst of requests; some return 429 Too Many Requests
WHAT I SAY	"Kuadrant and Limitador protect the platform from abuse. Rate limits are policy-as-code at the gateway."
EXPECTED RESULT	Mix of 200 and 429 under load

Story in one sentence

Preflight → bootstrap MaaS → subscribe + authorize → OpenAI API through the gateway → prove 403 without policy → prove rate limits.

No extra GPU. Pre-existing models in other namespaces are untouched.

Cleanup

```
make cleanup
```

See docs/TROUBLESHOOTING.md for common issues.